Universidad del Cauca
Facultad de Ingeniería Electrónica y Telecomunicaciones
Programa de Ingeniería de Sistemas

Banco de Preguntas Saber

Documento de arquitectura
Primer corte – Iteración 1

Equipo “Sin Nombre”

Integrantes: Juan Manuel Meneses Daza, Juan Sebastian Muñoz Ruiz, Juan Sebastian Molano Bolaños, Santiago Rodríguez Bolaños.

Ingeniería de Software II – 2026.2

Popayán, octubre de 2026

Índice

1. Resumen **3**

2. Requisitos funcionales **3**

2.1. Épicas . . . 3

2.2. Historias de usuario implementadas . . . 3

2.3. Alcance y limitaciones de la iteración . . . 5

3. Prototipos y test de usabilidad **5**

3.1. Test de usabilidad (thinking aloud) . . . 9

4. Requisitos no funcionales: escenarios de calidad **12**

4.1. Modificabilidad: nueva regla de validación . . . 12

4.2. Modificabilidad: sustitución de persistencia . . . 12

4.3. Seguridad . . . 13

5. Diagrama entidad–relación **13**

6. Arquitectura del sistema **14**

6.1. Modelo C4 . . . 14

6.1.1. Nivel 1: Contexto . . . 14

6.1.2. Nivel 2: Contenedores . . . 15

6.1.3. Nivel 3: Componentes . . . 16

6.1.4. Nivel 4: Código (UML de clases) . . . 17

6.2. Vistas 4+1 . . . 17

7. Decisiones de arquitectura y su justificación **18**

7.1. Selección del tipo de aplicación . . . 18

7.2. Selección del estilo arquitectónico . . . 19

7.3. Selección de tecnologías . . . 19

7.4. Patrones de diseño aplicados . . . 20

7.5. Elementos descartados . . . 20

8. Pruebas unitarias automatizadas **21**

9. Gestión del proyecto y planificación (Sprint 1) **21**

10.Sustentación y repositorio del proyecto **22**

1. Resumen

Este documento presenta la arquitectura del Banco de Preguntas Saber Pro, una aplicación de escritorio en Java (JavaFX y SQLite) para gestionar el ciclo de vida de preguntas de selección múltiple con única respuesta destinadas a la preparación de los estudiantes de Ingeniería de Sistemas. La primera iteración es un monolito en capas con el micro patrón MVC, y aplica los patrones Observer, Pipeline (tuberías y filtros) y State (máquina de estados) junto con principios SOLID, con inversión de dependencias como eje central.

La iteración incluye la gestión de usuarios (registro, inicio de sesión y listado, con contraseñas cifradas con Argon2id); la creación y modificación de preguntas con todos sus campos mediante un editor gráfico; la búsqueda de preguntas por criterios; la gestión del ciclo de vida de las preguntas con una máquina de siete estados, la auditoría de cada cambio de estado y vistas que se actualizan automáticamente; y la validación estructural de cada borrador mediante un pipeline de siete filtros, integrado al guardar. El documento describe los requisitos, los escenarios de calidad, el modelo de datos, la arquitectura con el modelo C4 y las decisiones técnicas con su justificación.

2. Requisitos funcionales

2.1. Épicas

- **HE1 – Gestión de usuarios:** registro, autenticación y listado de usuarios con roles.
- **HE2 – Gestión del banco de preguntas:** creación, modificación, consulta con filtros y gestión del estado de las preguntas.
- **HE3 – Validación estructural:** verificación de que cada pregunta cumple el formato requerido.

2.2. Historias de usuario implementadas

HU01 – Gestión de usuarios del sistema. **Como** administrador **quiero** registrar usuarios e iniciar sesión **para** controlar quién usa el sistema y con qué rol.

1. El registro solicita login, nombre completo, rol, estado (activo o inactivo) y contraseña; el login es único.
2. Los roles disponibles son administrador, autor de preguntas, revisor, docente y estudiante.
3. La contraseña cumple la política: mínimo 6 caracteres, un dígito, una mayúscula y un carácter especial.
4. La contraseña se almacena con Argon2id, nunca en texto plano.

5. El inicio de sesión verifica las credenciales contra la contraseña cifrada y se pueden consultar los usuarios registrados.

HU02 – Gestión del banco de preguntas. Como autor de preguntas **quiero** crear, modificar y buscar preguntas de selección múltiple **para** alimentar el banco de preguntas Saber Pro.

1. Desde el banco de preguntas se abre un editor para crear una pregunta nueva o modificar una existente.
2. La pregunta incluye nombre, competencia, tema, subtema, dificultad (baja, media o alta), contexto, pregunta directa, cinco opciones con una marcada como correcta (las otras cuatro son distractores), justificación y bibliografía.
3. Al guardar, el sistema aplica la validación estructural (HU03); si falla, no se guarda y se informan los errores.
4. Una pregunta nueva queda en estado Borrador.
5. Solo se pueden modificar preguntas en estado Borrador.
6. Las preguntas se pueden buscar por estado, dificultad, competencia, tema, subtema y texto.

HU04 – Gestión del ciclo de vida (estado de las preguntas). Como autor, revisor o administrador **quiero** consultar una pregunta y cambiar su estado **para** llevarla por su ciclo de vida.

1. Al seleccionar una pregunta se muestran sus datos, su estado actual y el historial de cambios de estado.
2. El banco define siete estados: Borrador, Pendiente de revisión, En revisión, Aprobada, Rechazada, Publicada y Archivada.
3. Las transiciones permitidas forman una máquina de estados; `QuestionState` declara las de cada estado y `Question.changeState` solo acepta las válidas. La interfaz ofrece únicamente el estado actual y sus transiciones válidas.
4. Cada cambio de estado se registra en el historial con el usuario que lo realizó y la fecha. No hay borrado físico: una pregunta se archiva, y Archivada es un estado final.
5. Cada cambio notifica automáticamente a dos vistas: el conteo de preguntas por estado y un gráfico circular con los porcentajes, con un color fijo por estado.
6. El acceso al banco de preguntas está disponible para los roles administrador, autor y revisor.

HU03 – Validación estructural. Como sistema **quiero** validar cada borrador de pregunta al crearlo o modificarlo **para** garantizar el formato requerido.

Orden	Filtro (requisito)	Regla
1	`ContextFilter` (RF-08)	Toda pregunta tiene contexto (mínimo 20 caracteres).
2	`DirectQuestionFilter` (RF-09)	Una única pregunta directa, con formato de pregunta.
3	`DistractorCountFilter` (RF-10)	Exactamente cuatro distractores (cinco opciones con la correcta).
4	`ForbiddenExpressionFilter` (RF-12)	Se prohíben "Todas las anteriores" y "Ninguna de las anteriores".
5	`DistractorFormatFilter` (RF-13)	Opciones no vacías, de al menos 3 caracteres, con mayúscula inicial y sin repetirse.
6	`CorrectAnswerFilter` (RF-11)	Una única respuesta correcta, con índice válido.
7	`RequiredFieldsFilter` (RF-05)	Campos obligatorios: nombre, justificación, bibliografía, competencia, tema, subtema y nivel de dificultad.

Tabla 1: Filtros del pipeline de validación estructural.

Si un filtro falla, el pipeline se detiene y lanza una `ValidationException` con los errores. `QuestionService` ejecuta el pipeline en `createQuestion` y `updateQuestion`.

2.3. Alcance y limitaciones de la iteración

Las siguientes funcionalidades no están implementadas en esta iteración y quedan previstas para las siguientes:

- Asignación de revisores por el administrador y notificación por correo electrónico.
- Paginación del listado de preguntas (hoy se filtra, pero no se pagina).
- Registro del autor de cada pregunta (el historial de cambios de estado ya se audita con usuario y fecha).

3. Prototipos y test de usabilidad

A continuación se presentan los wireframes de las interfaces de la aplicación de escritorio JavaFX:



Figura 1: Wireframe de la vista de inicio de sesión.



Figura 2: Wireframe de la vista de registro de usuarios.

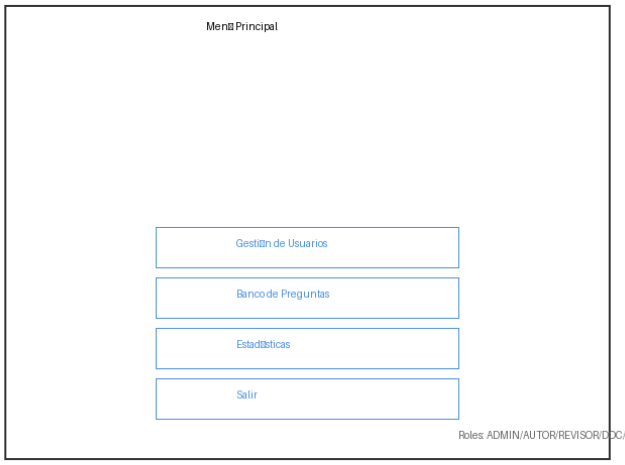


Figura 3: Wireframe del menú principal según el rol del usuario.

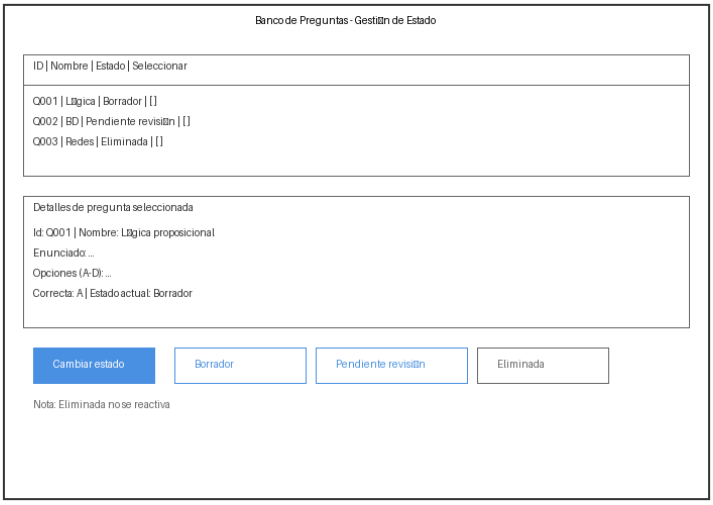


Figura 4: Wireframe del banco de preguntas y gestión de estados.

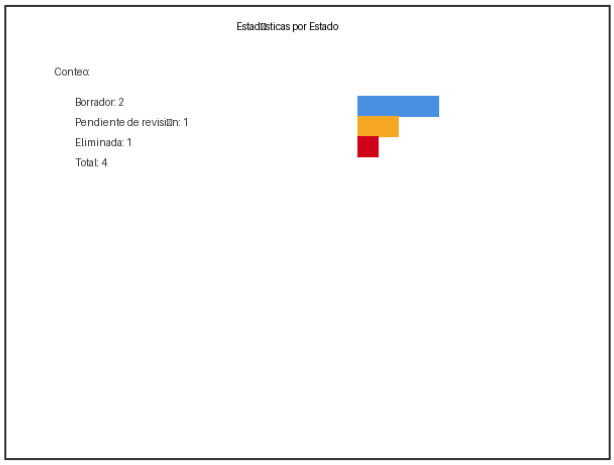


Figura 5: Wireframe del módulo de estadísticas y conteo por estado.

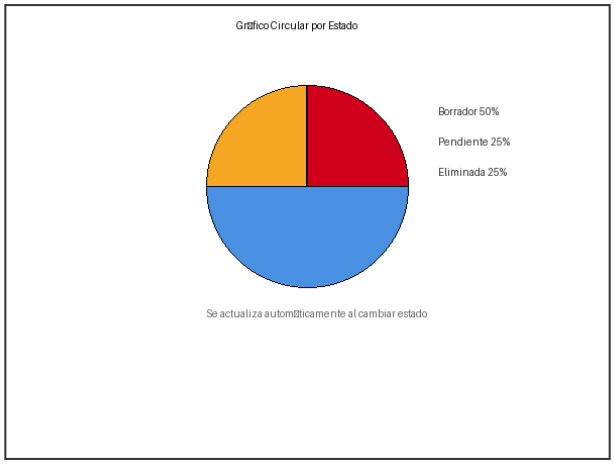


Figura 6: Wireframe del gráfico circular.

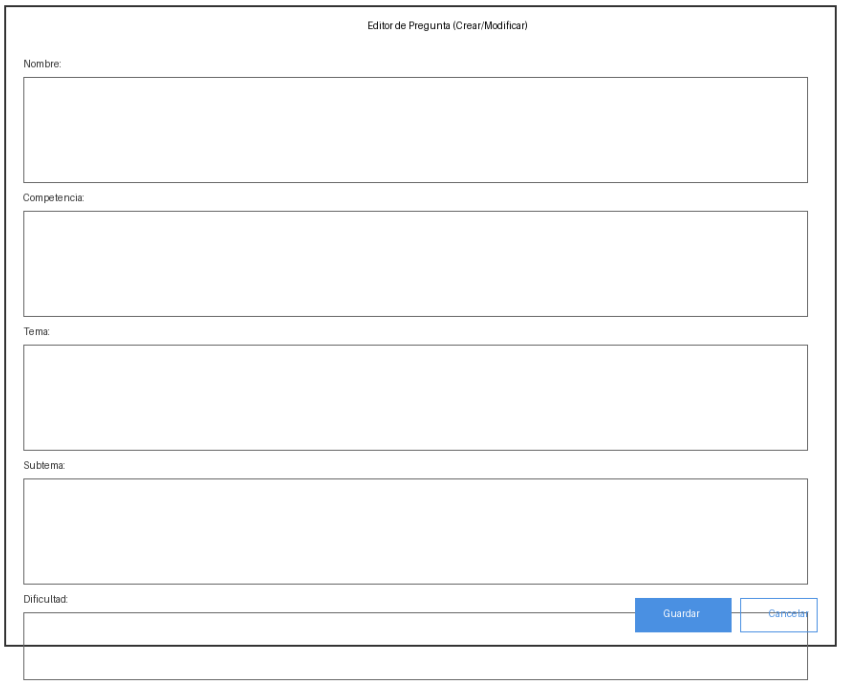


Figura 7: Wireframe del editor para crear y modificar preguntas.

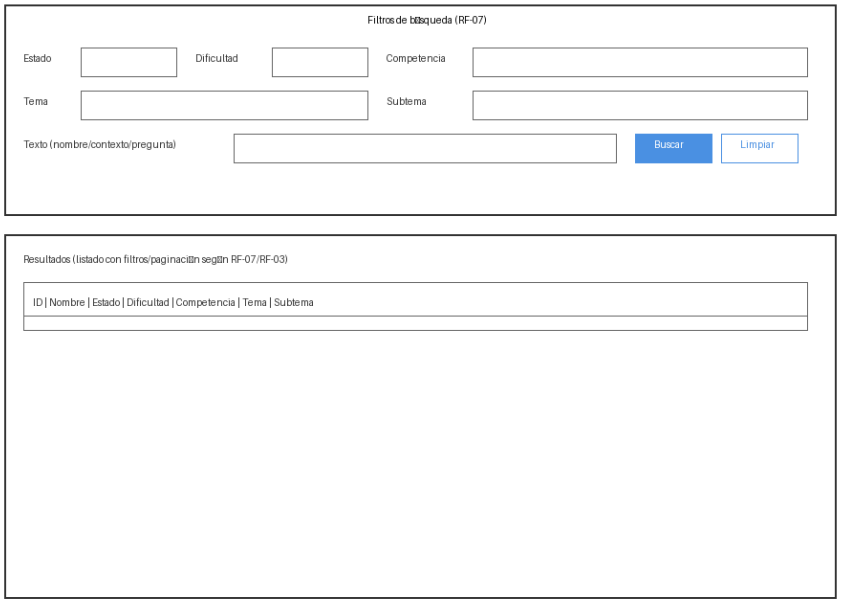


Figura 8: Wireframe de la búsqueda y filtros de preguntas.

3.1. Test de usabilidad (thinking aloud)

Método. Se pide a cada participante que ejecute tareas concretas con la aplicación mientras verbaliza en voz alta lo que piensa, espera y no entiende. Un observador registra los

comentarios, los errores, las dudas y el tiempo, sin ayudar al participante.

Participantes: Tres estudiantes de Ingeniería de Sistemas. Dos participantes simularon el rol de autor de preguntas y uno el de administrador. Los participantes no pertenecían al equipo de desarrollo, con el fin de observar las dificultades de personas que no conocían directamente la implementación interna del sistema.

Fecha y lugar: Octubre de 2026, Universidad del Cauca, Popayán.

Versión probada: Prototipo funcional de la primera iteración del Banco de Preguntas Saber Pro.

Tarea	**Criterio de éxito**	**Resultado observado**	**Hallazgos y mejoras**
Registrarse e iniciar sesión (HU01)	El usuario queda registrado e inicia sesión con una contraseña que cumple la política	El participante comprendió los campos del formulario, pero durante el registro intentó utilizar una contraseña que no cumplía todos los requisitos. Después de recibir el mensaje de validación, tuvo que revisar nuevamente la contraseña y corregirla. También manifestó dudas sobre qué caracteres especiales estaban permitidos.	Mejorar la visibilidad de la política de contraseñas junto al campo correspondiente. Mostrar los requisitos de manera individual y señalar cuáles se cumplen mientras se escribe.
Crear una pregunta con el editor (HU02)	La pregunta se guarda en Borrador, o se entienden los errores de validación	El participante comenzó a diligenciar el formulario, pero intentó guardar la pregunta antes de completar la bibliografía y la justificación. La aplicación rechazó el registro y mostró errores de validación. Aunque pudo identificar que faltaban datos, tuvo dificultades para determinar cuáles campos debía corregir primero. También necesitó revisar las cinco opciones de respuesta para confirmar cuál estaba marcada como correcta.	Agrupar visualmente los campos obligatorios, destacar los errores junto a cada campo y presentar un resumen de las validaciones fallidas. Diferenciar con mayor claridad la respuesta correcta de los distractores.
Buscar con filtros (HU02)	Se encuentra una pregunta usando al menos un filtro	El participante intentó localizar una pregunta utilizando simultáneamente los criterios de competencia y dificultad. Inicialmente no tuvo claro si debía seleccionar cada criterio antes de ejecutar la búsqueda o si los resultados se actualizaban automáticamente. Tras interactuar con los controles, logró localizar la pregunta,	Aclarar el funcionamiento de los filtros mediante etiquetas descriptivas y un botón de búsqueda claramente identificable. Incorporar una opción visible para limpiar los criterios seleccionados.

Una de las principales dificultades se presentó durante la creación de preguntas. Los campos obligatorios, la política de contraseñas y las reglas asociadas a las opciones de respuesta requieren una comunicación clara para evitar que el usuario deba descubrirlas mediante intentos sucesivos. Asimismo, la gestión de estados evidenció la importancia de proporcionar confirmaciones visuales que permitan distinguir entre una acción solicitada y una modificación efectivamente reflejada en la interfaz.

4. Requisitos no funcionales: escenarios de calidad

4.1. Modificabilidad: nueva regla de validación

Fuente	Desarrollador del equipo.
Estímulo	Se requiere agregar una nueva regla de validación estructural de preguntas.
Artefacto	`QuestionPipeline` y la interfaz `Filter` (paquete `pipeline`).
Entorno	Tiempo de desarrollo, sistema en su versión actual.
Respuesta	Se crea un filtro que implementa `Filter` y se agrega a la lista del pipeline; los filtros y servicios existentes no se modifican.
Medición	Una clase nueva y su clase de prueba; una línea de registro en `QuestionPipeline`; las pruebas existentes permanecen en verde; tiempo estimado menor a 2 horas.
Resultado esperado	La regla queda activa en la validación sin regresiones.

Tabla 3: Escenario de modificabilidad.

4.2. Modificabilidad: sustitución de persistencia

Fuente	Desarrollador del equipo.
Estímulo	Se desea cambiar SQLite por otro motor (MariaDB o Postgres).
Artefacto	Capa `access` (`IQuestionRepository`, `IUserRepository`).
Entorno	Tiempo de desarrollo.
Respuesta	Se escriben nuevas implementaciones de las interfaces y se cambia la composición en `MainApp`.
Medición	Cero cambios en `service`, `domain` y `ui`, que dependen de abstracciones.
Resultado esperado	La aplicación funciona con el nuevo motor y las pruebas de servicio siguen en verde.

Tabla 4: Segundo escenario de modificabilidad.

4.3. Seguridad

Las contraseñas se almacenan con Argon2id, existe una política mínima de contraseña y el acceso al banco de preguntas se restringe por rol en el menú principal.

5. Diagrama entidad–relación

El esquema relacional se crea en SQLite desde `DatabaseConnection`.

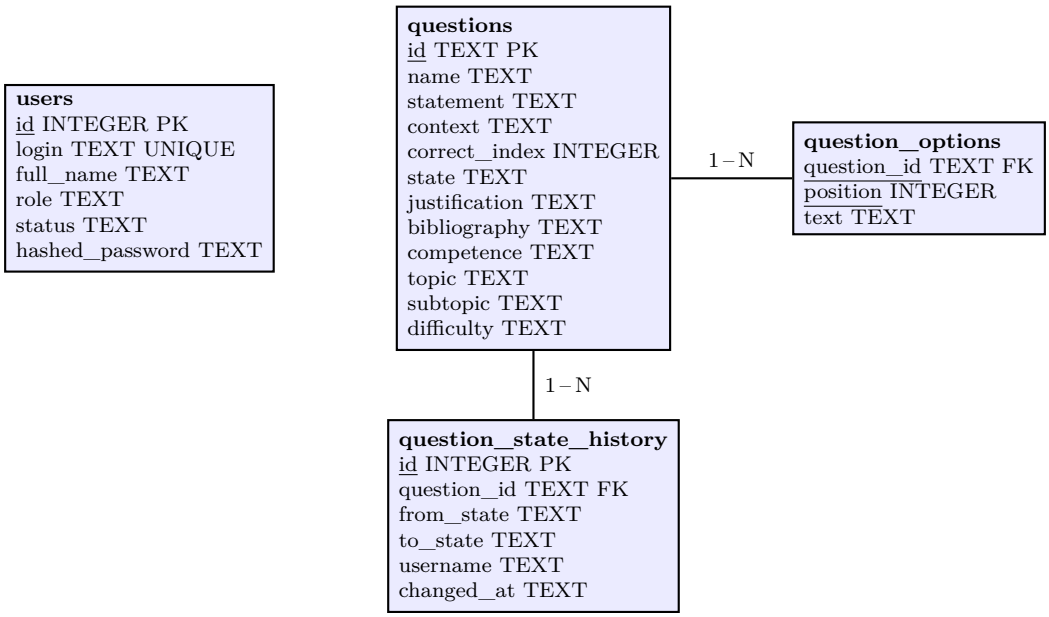


Figura 9: Modelo de datos actual.

La tabla `questions` se relaciona con `question_state_history`, que conserva la auditoría de cada cambio de estado: estado de origen y de destino, usuario y fecha. La tabla `users` aún no se relaciona con `questions`: no se registra el autor de cada pregunta, por lo que el usuario del historial se guarda como texto. Es una limitación conocida para la trazabilidad en iteraciones posteriores.

6. Arquitectura del sistema

6.1. Modelo C4

6.1.1. Nivel 1: Contexto

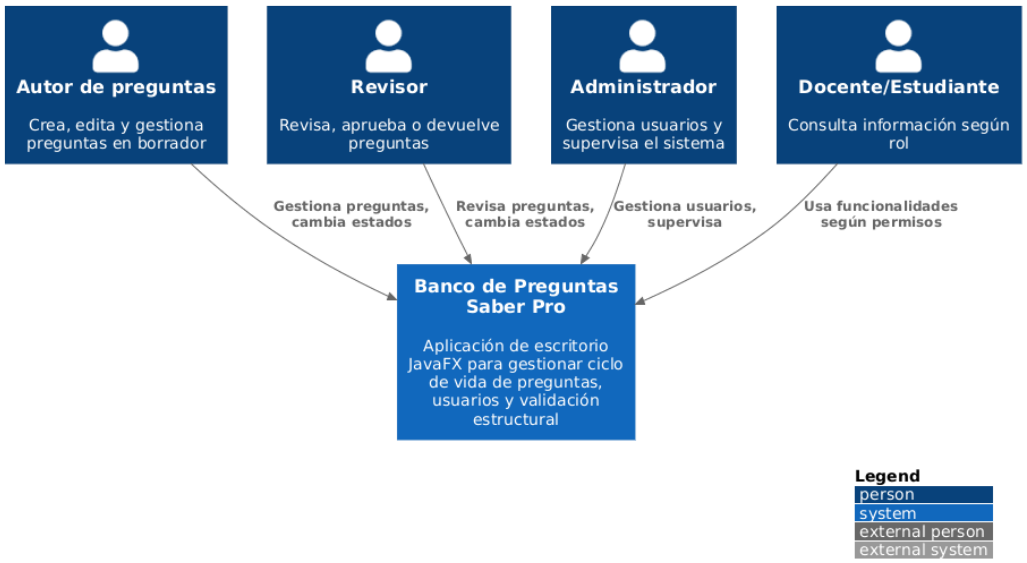


Figura 10: Diagrama C4 - Nivel 1: Contexto del sistema.

6.1.2. Nivel 2: Contenedores

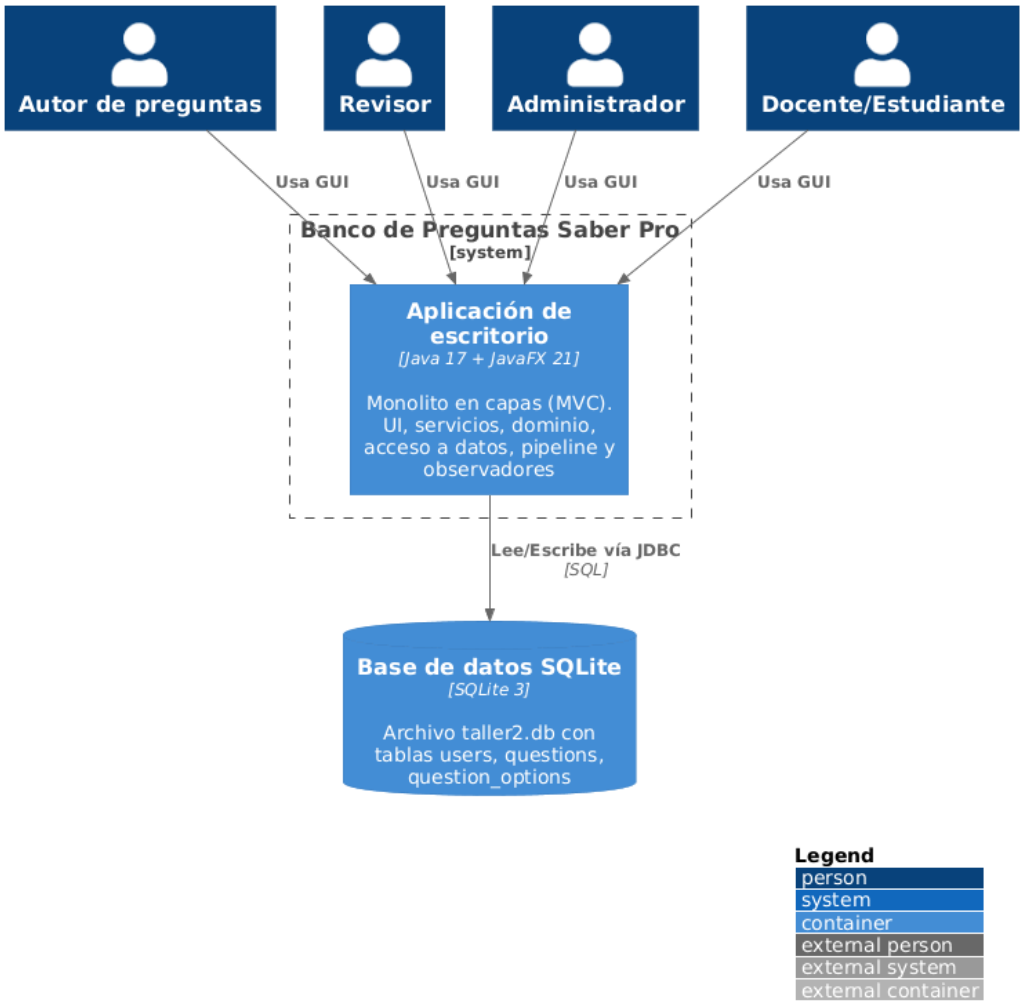


Figura 11: Diagrama C4 - Nivel 2: Contenedores.

6.1.3. Nivel 3: Componentes

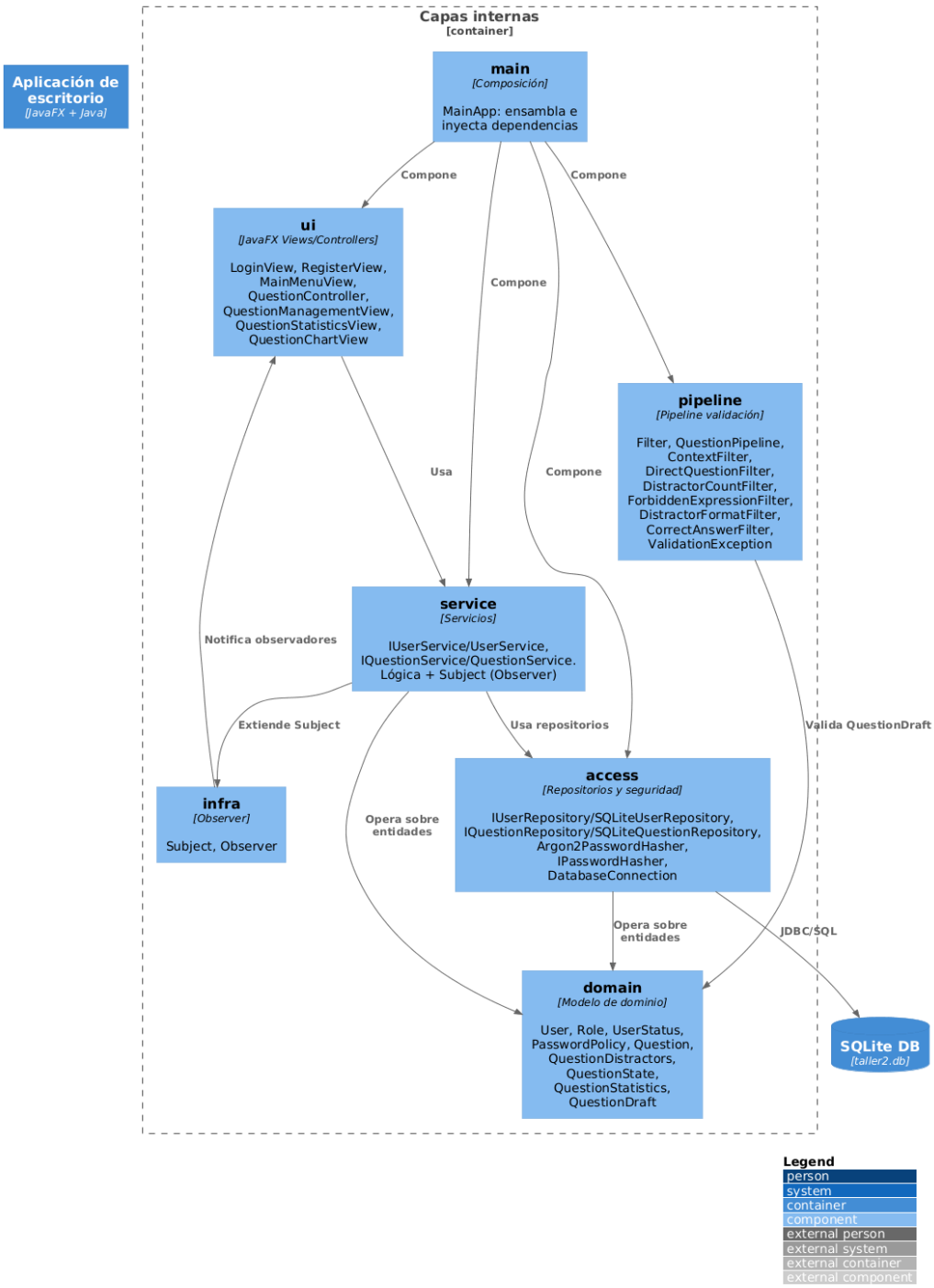


Figura 12: Diagrama C4 - Nivel 3: Componentes del monolito en capas.

6.1.4. Nivel 4: Código (UML de clases)

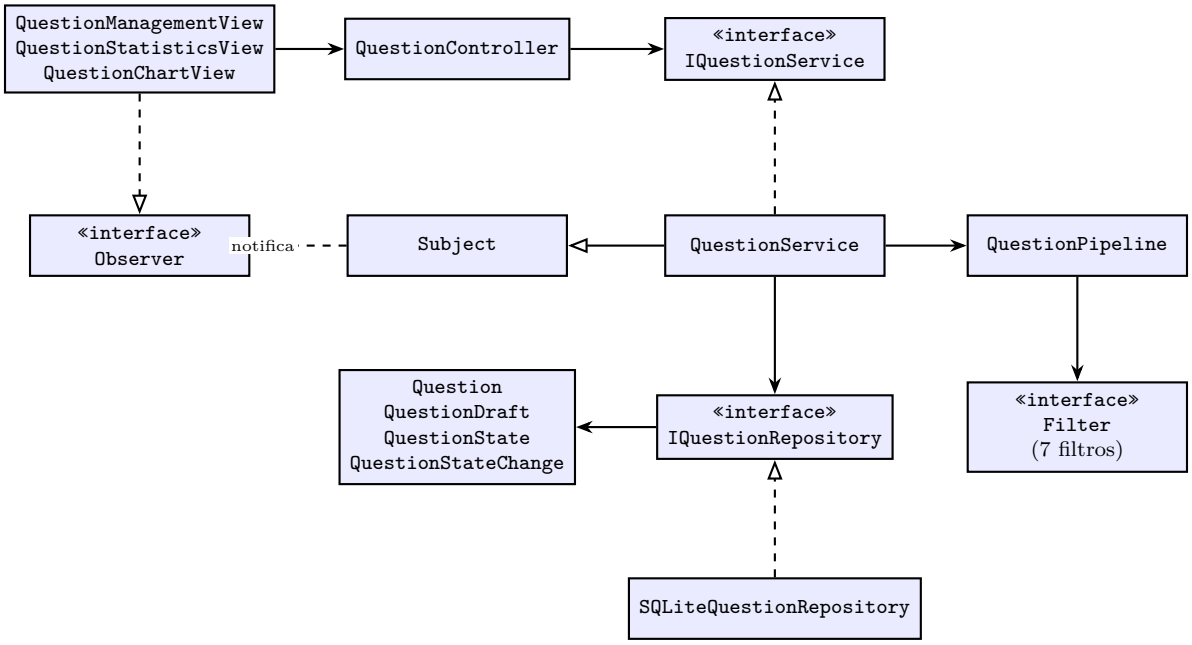


Figura 13: Diagrama de clases simplificado del banco de preguntas (las flechas punteadas con triángulo indican implementación).

6.2. Vistas 4+1

- **Lógica:** paquetes `domain`, `service`, `pipeline`, `access`, `infra` y `ui` (figura del nivel 4).
- **Desarrollo:** organización por paquetes y responsabilidades; `main/MainApp` compone las dependencias.
- **Procesos:** secuencia de "crear pregunta" (siguiente figura).
- **Física/despliegue:** aplicación de escritorio en JVM 17 con JavaFX 21 y el archivo `taller2.db` de SQLite en la misma máquina.
- **Escenarios:** HU01, HU02, HU03 y HU04.

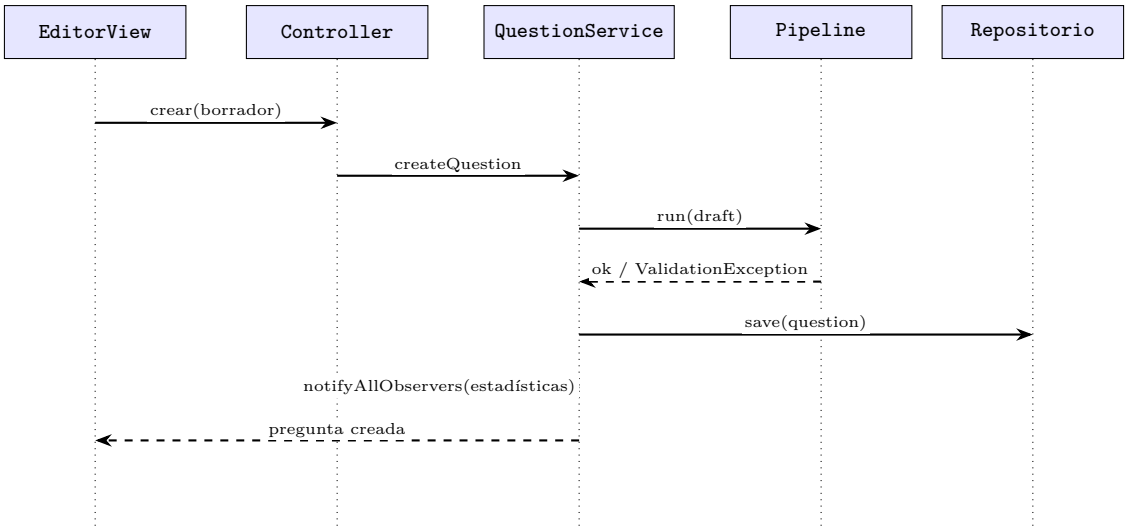


Figura 14: Vista de procesos: crear una pregunta con validación estructural y notificación a los observadores.

7. Decisiones de arquitectura y su justificación

Aspecto	Decisión	Alternativas no elegidas
Tipo de aplicación	Escritorio (JavaFX)	Web, móvil, híbrida
Estilo	Monolito en capas con micro patrón MVC	Microservicios
Plataforma	Java 17 (Java SE)	Jakarta EE, Spring Boot
Base de datos	SQLite (JDBC 3.45.3.0)	MariaDB, Postgres
Seguridad	Argon2id (argon2-jvm 2.11)	Texto plano, hashes rápidos
Construcción	Maven	Gradle, Ant
Pruebas	JUnit 5.10.2 con dobles escritos a mano y Mockito 5.23.0	TestNG
Gestión	Jira y GitHub	Trello, GitHub Projects
Patrones	Observer, Pipeline, State (máquina de estados), MVC, DIP	–

Tabla 5: Resumen de decisiones técnicas.

7.1. Selección del tipo de aplicación

Se eligió una aplicación de escritorio con JavaFX, una de las opciones que sugiere el enunciado. **Ventajas:** no requiere servidor ni despliegue web, se ejecuta con `mvn javafx:run` y permite concentrar el esfuerzo en las capas, los patrones y las pruebas. **Desventajas:** el acceso queda limitado a la máquina donde se instala, no hay concurrencia real entre usua-

rios y la evolución hacia una solución distribuida (iteración 2) exigirá separar la interfaz del servidor.

7.2. Selección del estilo arquitectónico

Monolito en capas con micro patrón MVC. El paquete `ui` contiene las vistas y el controlador (`QuestionController`), y `QuestionService` actúa como modelo y sujeto observable. Las dependencias fluyen de `ui` a `service`, `access` y `domain`, y las capas superiores dependen de abstracciones (`IUserService`, `IUserRepository`, `IPasswordHasher`, `IQuestionService`, `IQuestionRepository`); las implementaciones se inyectan en `MainApp`.

Ventajas: separación de responsabilidades, bajo acoplamiento, pruebas con dobles y coherencia con el requisito del curso de empezar con un monolito en capas (MVC) y refactorizar a microservicios después. **Desventajas:** más clases e interfaces para un alcance pequeño, todo se despliega junto y no se escala por módulo. Se mitiga manteniendo las capas desacopladas por interfaces.

Por qué no microservicios: el sistema tiene pocas épicas, un equipo pequeño y ningún módulo que requiera escalar de forma independiente; su costo de comunicación, serialización y versionado de contratos no se justifica en esta iteración.

7.3. Selección de tecnologías

Java 17 y JavaFX 21. Plataforma que domina el equipo; JavaFX ofrece componentes de interfaz y gráficos (el gráfico circular de estados). Hay que escribir a mano la composición de dependencias, sin un contenedor de inyección.

SQLite. Base de datos embebida en un archivo, sin instalación y con modo en memoria para probar el repositorio. **Desventaja:** soporta mal la concurrencia y el escalamiento (el enunciado pide al menos cien usuarios concurrentes), por lo que habría que migrar a MariaDB o Postgres; la capa `access` basada en interfaces limita ese cambio a nuevas implementaciones.

Argon2id. Algoritmo de hash recomendado para contraseñas, que cumple el requisito de almacenarlas con hash seguro. Añade una dependencia nativa.

Maven. Estructura estándar y dependencias declarativas; ejecuta la aplicación con `mvn javafx:run` y las pruebas con `mvn test`. Su `pom.xml` es verboso.

JUnit 5 y dobles de prueba. Las pruebas usan JUnit 5. La mayoría de los dobles son escritos a mano (`FakeUserRepository`, `FakeQuestionRepository`, `FakePasswordHasher`); además, Mockito 5.23.0 genera dobles sobre las abstracciones (`IQuestionRepository` e

`IQuestionService`) en los casos que verifican interacciones. **Desventaja:** los dobles manuales deben mantenerse al cambiar las interfaces.

Jira y GitHub. Jira da trazabilidad de historias y tareas, y GitHub aporta ramas, control de versiones y evidencia del aporte de cada integrante. Se mitiga la carga administrativa con un flujo mínimo de estados.

7.4. Patrones de diseño aplicados

Patrón	Dónde se aplica	Para qué
Capas y DIP	Paquetes `ui`, `service`, `access`, `domain`; `MainApp` como raíz de composición	Bajo acoplamiento y sustitución de implementaciones.
MVC (micro patrón)	`QuestionController`, vistas del banco y `QuestionService` como modelo	Separar interfaz, control y lógica.
Observer	`QuestionService` extiende `Subject`; las vistas de estadísticas y gráfico implementan `Observer`	Actualizar las vistas al cambiar los datos sin que el servicio las conozca.
Pipeline (tuberías y filtros)	`QuestionPipeline` y siete filtros que implementan `Filter`	Validar la estructura de cada pregunta con reglas independientes.
State (máquina de estados)	`QuestionState`: cada constante conoce sus transiciones válidas y `Question.changeState` las aplica	Restringir el ciclo de vida de la pregunta (HU04) a transiciones válidas.

Tabla 6: Patrones de diseño implementados.

Observer. El riesgo es que las notificaciones sean difíciles de rastrear si crece el número de observadores; `Subject` ignora nulos y duplicados al registrar. **Pipeline.** Se pueden agregar o quitar reglas sin tocar las demás (abierto/cerrado); `QuestionPipeline` también admite una lista de filtros propia. El orden de los filtros afecta qué error se informa primero. **State.** Cada estado encapsula sus transiciones, de modo que agregar un estado nuevo (y sus reglas) no obliga a tocar las entidades ni el servicio que lo usan.

7.5. Elementos descartados

- **Spring Boot:** oculta la inyección y la configuración, que se quiere practicar a mano.
- **Microservicios:** complejidad innecesaria en la iteración 1; se abordan en la iteración 2.

8. Pruebas unitarias automatizadas

Las pruebas se ejecutan con `mvn test`. El proyecto tiene 152 casos de prueba en 22 clases que cubren entidades y objetos de dominio (`User`, `PasswordPolicy`, `Question`, `QuestionState`, `QuestionDistractors`, `QuestionCriteria`, `QuestionStatistics`), los servicios de usuario y de preguntas (incluida la auditoría de los cambios de estado), el controlador, `Subject`, los siete filtros y el pipeline de punta a punta, y el repositorio SQLite sobre una base en memoria, donde además se comprueba la persistencia del historial de cambios de estado. La mayoría de las pruebas usan dobles escritos a mano y nueve casos usan Mockito para falsificar `IQuestionRepository` y `IQuestionService`, verificar las interacciones y capturar el contenido de la auditoría.

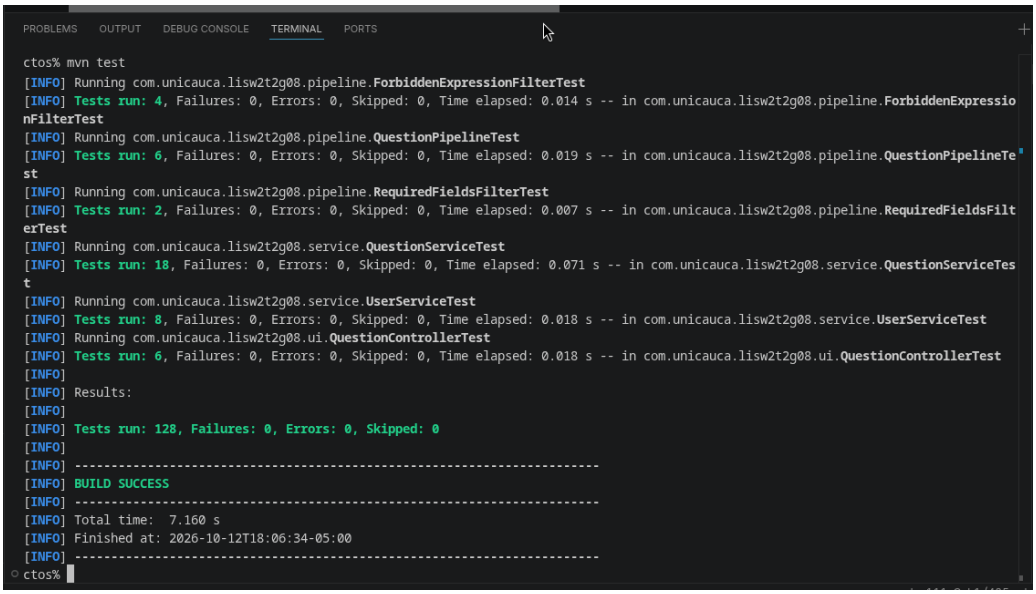


Figura 15: Ejecución exitosa de las pruebas unitarias automatizadas en Maven.

9. Gestión del proyecto y planificación (Sprint 1)

Para la gestión del Sprint 1, el equipo utilizó **Jira** y **GitHub**. A continuación se presenta la captura del tablero de planificación de tareas:

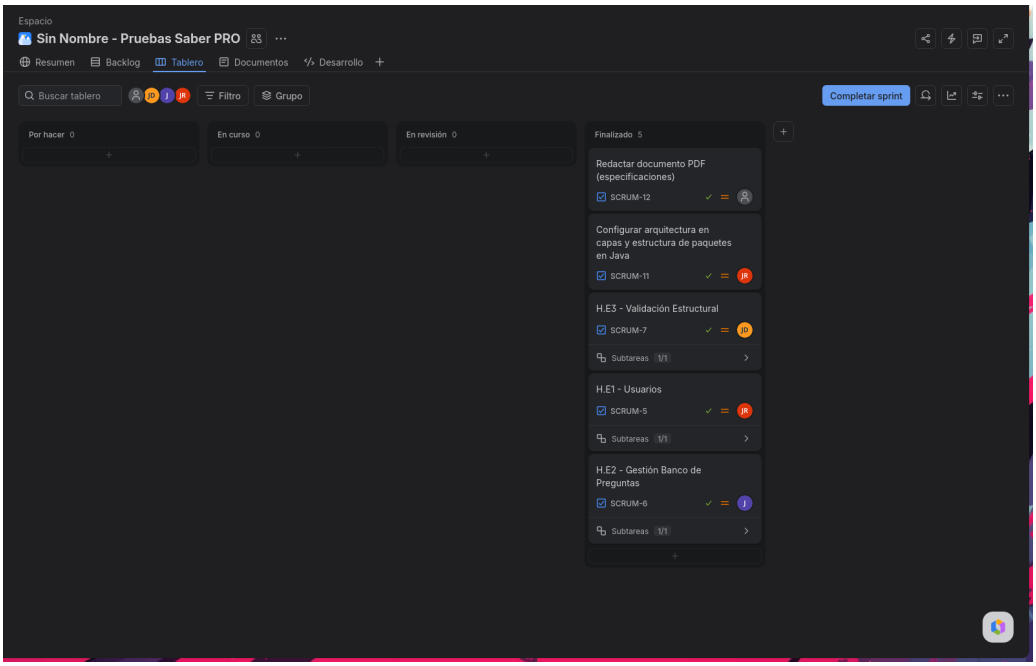


Figura 16: Tablero de planificación de tareas del Sprint 1 en Jira.

Tablero: https://unicauca-team-vpqtq3w8.atlassian.net/

10. Sustentación y repositorio del proyecto

- **Video (sustentación): https://youtu.be/DqOJavD97cc**
- **URL del repositorio Git (GitHub):** https://github.com/BastianRu/proyecto-saberpro-i